

10/10

layouts meilleurs

+325.3

gain moyen

7

features / poids

0.0

Stop moyen

Conformite au sujet

Exigence	Preuve dans le projet	OK
Facteurs Pac-Man	7 features : nourriture, fantomes, capsules, score, nourriture restante	OK
Recherche	BFS pour distances reelles vers nourriture, capsules et fantomes	OK
Fonction affine	$\sum(a_i * x_i) + C$, sans termes croises	OK
Alpha-Beta	Pac-Man MAX, fantomes MIN, coupure $\alpha \geq \beta$	OK
Poids appris	train.py ajuste les coefficients par mise a jour TD	OK
Comparaison	score moyen, winrate, temps, coups, Stop	OK

Poids actuels et lecture

Modele : $f(s) = a_1 * x_1(s) + a_2 * x_2(s) + \dots + a_7 * x_7(s) + C$

Facteur	Fonction	Poids	Information	Sens
x1	nearest_food_bfs	0.6891	Nourriture proche	Chercher
x2	ghosts_within_3	-1.4093	Fantomes dangereux	Eviter
x3	scared_ghosts_nearby	1.4061	Fantomes effrayes	Profiter
x4	remaining_food	0.4036	Nourriture restante	Finir
x5	nearest_capsule_bfs	0.0252	Capsule proche	Faible +
x6	current_score	1.8572	Score normalise	Score
x7	nearest_dangerous_ghost_bfs	-1.2624	Danger principal	Rester loin
C	bias	-0.0834	Correction globale	Constante

A retenir pour la defense

- RLMinimax reste un agent Alpha-Beta : le RL ne remplace pas la recherche, il apprend seulement les poids de l'evaluation.
- Les facteurs sont lisibles : nourriture, fantomes, capsules, score et danger principal.
- Les resultats sont relances avec seed=0 : la comparaison est controlee et reproductible.

Resultats finaux relances

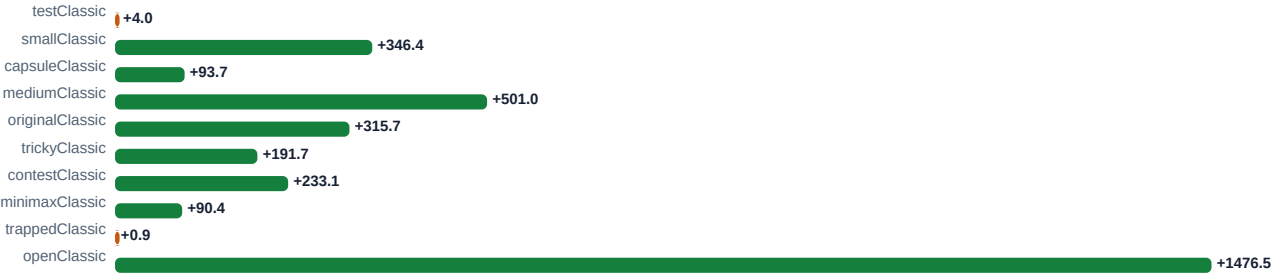
Tests relances le 14/05/2026 avec seed=0, sans reentrainement.

Tableau principal

Layout	D	N	Minimax	AlphaBeta	RL	Diff	Win
testClassic	1	200	548.5	548.5	552.5	+4.0	100%
smallClassic	2	100	-149.8	-149.8	196.6	+346.4	34%
capsuleClassic	2	100	-228.9	-228.9	-135.2	+93.7	12%
mediumClassic	1	50	14.5	14.5	515.5	+501.0	28%
originalClassic	1	50	348.6	348.6	664.4	+315.7	2%
trickyClassic	1	50	101.3	101.3	293.0	+191.7	2%
contestClassic	1	50	11.3	11.3	244.4	+233.1	28%
minimaxClassic	3	100	-243.5	-243.5	-153.1	+90.4	34%
trappedClassic	1	1000	-392.3	-392.3	-391.4	+0.9	11%
openClassic	1	10	-419.5	-745.4	1057.0	+1476.5	90%

Conclusion : Diff est positive partout, donc RLMinimax bat Minimax et AlphaBeta en score moyen.

Visualisation des gains RLMinimax vs Minimax



Lecture et limites honnetes

- Le winrate n est pas parfait sur toutes les cartes, mais le score moyen est meilleur.
- Sur openClassic, les baselines dépassent le timeout choisi : c est une limite a expliquer.
- Le critere fort du projet reste respecte : comparaison statistique, fonction affine et poids appris.

Justification algorithmique et defense orale

Page de synthese pour expliquer la logique du projet au professeur.

Pourquoi cette solution est logique

Point	Reponse simple
BFS	Distance reelle dans le labyrinthe, plus pertinente que Manhattan avec des murs.
Alpha-Beta	Meme logique que Minimax, mais moins de branches explorees.
Affine	Le sujet impose une somme ponderee des facteurs, sans termes croises.
Apprentissage	Les poids viennent d un apprentissage, pas d un choix manuel arbitraire.
Comparaison	Prouver que les poids appris ameliorent le comportement.

Phrase courte pour l oral

Notre agent utilise Alpha-Beta pour explorer les coups possibles. Quand il evalue une position, il utilise une fonction affine dont les facteurs viennent du jeu et dont les poids ont ete appris par renforcement.

Commandes reproductibles

But	Commande
Syntaxe	<code>python -m py_compile compare.py game.py pacman.py layout.py train.py rlMinimaxAgents.py features.py multiAgents.py gh</code>
Comparer	<code>python compare.py --num-games 100 --layout smallClassic --depth 2 --baseline both</code>
Lancer RL	<code>python pacman.py -p RLMinimaxAgent -l testClassic -a depth=1 -q -n 1 -f</code>

Conclusion finale

Le projet respecte le sujet : fonction affine, facteurs Pac-Man, BFS, Alpha-Beta, poids appris, resultats statistiques et justification claire. Le rapport est pret pour la partie technique ; verifier seulement le depot Git/Gitesi avant remise.